Native hashed-ecdl-tensor measurements on small prime-order curves

Study: `research/btn_ecdl_prime_20261029/` (PR #2888). Tool: `native/` (`btn-prime-scaling`, Rust), depending on the unmodified `crates/hashed-ecdl-tensor` crate (#2878). All numbers below are read from `native_runs/report.json`, produced by `btn-prime-scaling report native_runs` from the raw records `sizes.jsonl`, `counts.jsonl`, `solves.jsonl` and `eccp131_size.json`.

Unit. Step counts are the crate's `Meter` charges, deterministic diagnostics and not calibrated time units. **No wall-clock time is reported**, so no timing isolation applies. Host: a 4-core Claude Code cloud container (Linux).

Exact counts against brute force

16 contractions of the decomposition network (two slots, full and hash-filtered factor bases) at p = 5, 7, 11, 13. All match brute force: **`true`**. Backend `Functional` in every count: **`true`**. Visited assignments equal 2^{4d} in every count: **`true`**. Recursive (OpenAI Appendix D) backend used anywhere: **`false`**.

p	d	brute	backend	assignments	steps	steps / assignment
5	3	match	Some(Functional)	4096	4485600	1095
5	3	match	Some(Functional)	4096	4485600	1095
5	3	match	Some(Functional)	4096	3523331	860
5	3	match	Some(Functional)	4096	3544187	865
7	3	match	Some(Functional)	4096	9627056	2350
7	3	match	Some(Functional)	4096	9627056	2350
7	3	match	Some(Functional)	4096	6463893	1578
7	3	match	Some(Functional)	4096	6463893	1578
11	4	match	Some(Functional)	65536	69188532	1056
11	4	match	Some(Functional)	65536	69188532	1056
11	4	match	Some(Functional)	65536	58303887	890
11	4	match	Some(Functional)	65536	58318774	890
13	4	match	Some(Functional)	65536	107725686	1644
13	4	match	Some(Functional)	65536	107725686	1644
13	4	match	Some(Functional)	65536	93262386	1423
13	4	match	Some(Functional)	65536	93277273	1423

End-to-end solves (`reduction::solve`, seeded)

p	d	order r	seed	secret	recovered	relations	steps
5	3	7	1	2	2 (verified)	13	210937575
5	3	7	2	1	1 (verified)	13	211864590
7	3	13	1	3	3 (verified)	25	738643764

Network size

d	p	edges	vertices	max degree
3	5	31624	8402	132
4	11	56480	12860	132
5	17	57952	13722	132
6	37	59580	14662	132
7	67	61120	15558	132
8	131	64848	16552	132
9	257	114192	19438	132
10	521	115816	20376	132

11	1031	117440	21314	132
12	2053	118916	22178	132
13	4099	120456	23074	132
14	8209	122008	23976	132
15	16411	123648	25182	260
16	32771	127920	31176	260
17	65537	223736	35474	260

ECCp-131 (d = 131, published challenge input, certified primes): 2 slots, 1684796 edges, 249620 vertices, max degree 1032, 526 interface bits (4 · 131 free coordinate bits plus 2 constant identity flags).

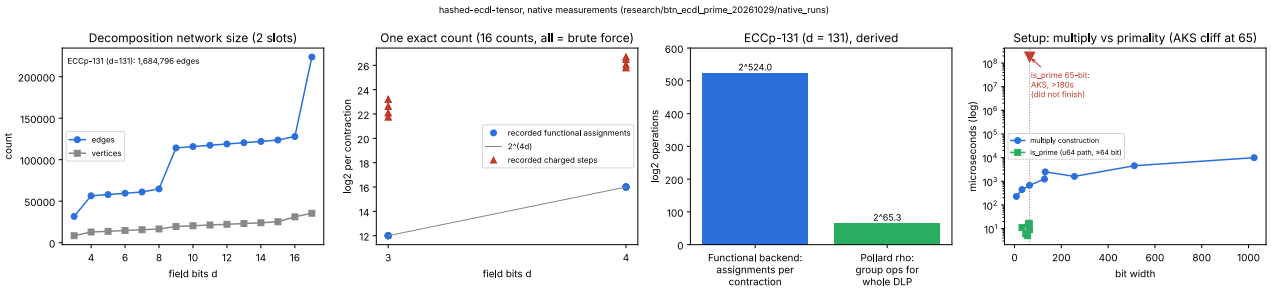


Figure 1: Recorded sizes, per-count assignments and steps, and the derived ECCp-131 comparison. Source: `native_runs/report.json`.

Setup: primality certification above 64 bits (resolved with certificates)

`is_prime` (used by `Curve::new` and `FactorBase::new`) dispatches to a fast u64 path at <= 64 bits and to **AKS** above.

Without certificates, every certification at <= 64 bits took at most 17 µs, while AKS at 65 bits did **not finish in 180 s**. That is why the first uncertified ECCp-131 build never finished.

The crate on `main` now accepts independently checked Pocklington certificates (`Curve::with_cofactor_and_certificates`, landed by a separate change). With the bundled `eccp131-certified-hash.json` (5 certificates, certified 132-bit hash prime) the ECCp-131 curve validates in about 13 ms and its full decomposition network builds (below). Network construction itself is fast at every width: the Fourier integer-multiply builds in 8-bit: 229 µs, 32-bit: 447 µs, 64-bit: 674 µs, 128-bit: 1244 µs, 131-bit: 2508 µs, 256-bit: 1610 µs, 512-bit: 4550 µs, 1024-bit: 9950 µs. So setup is no longer the wall; the recursion crossover is.

Speeding up the exact fallback that actually runs

At every reachable size the count runs on the exact `Functional` fallback, so that is where speed-ups land. Two were measured on the same 16 instances, in the crate's charged steps (deterministic `Meter` diagnostics, not time).

Upstream functional pruning (crate change landed separately, measured here): 16 of 16 instances paired, counts unchanged, charged steps 1.5–6.4× lower.

Derived last point (this study): for two slots the decomposition network searches both A_1 and A_2 as free inputs, but A_2 = target − A_1 is determined. Computing it with the encoder instead of searching it is a bijection on the same identity-free language, so counts cannot change, while the free input bits halve (d = 3: 12 ⟶ 6, d = 4: 16 ⟶ 8). Two refinements on top: mark only A_1 as the interface, so the crate's witness search may stop at its first hit (A_1 is the word's prefix and fixes A_2, so the canonical witness is unchanged); and check A_1's factor-base hash before building A_2 ("hash-first"), so an A_1 outside its base is rejected before the point addition runs. All variants are built only from the crate's public `Encoder` API; the crate is not modified.

Measured on 32 instances (p = 5, 7, 11, 13; two targets; hash thresholds τ = s, s/2, s/4, s/8, where s is the hash prime):

- Every variant's count equals the original network and brute force: **`true`**.
- Witnesses: on the fully marked derived network, 18 found and each checked by reference arithmetic and `FactorBase::contains`; on the A_1-only interface, all checked (**`true`**) and the same canonical witness in every solvable case (**`true`**).
- Combined (derived, A_1 interface, hash-first) versus the current pruned original: charged steps 1.5–65.3× lower, worse on 0 instances. Hash-first alone contributes 1–1.56×, largest at small τ.

- Limit: without hash-first, the derived network was worse than the original on 2 instances (p = 7, τ = 2), where the original's own pruning leaves few assignments and a fixed per-call overhead (about 84k charged steps at d = 3, 140k at d = 4, roughly 5 per edge) dominates. The gain is largest when many assignments survive, i.e. at large τ and larger p.

p	τ	count	steps original	steps derived	steps combined	combined ×
5	11	5	2379059	163933	163863	14.5×
5	11	5	2379059	163933	163863	14.5×
5	5	0	1314140	135637	124653	10.5×
5	5	2	1334996	146621	135609	9.8×
5	2	0	688359	134801	112833	6.1×
5	2	0	688359	134801	112833	6.1×
5	1	0	424002	133654	106194	4×
5	1	0	424002	133254	105794	4×
7	17	11	6563062	262515	262361	25×
7	17	11	6563062	262515	262361	25×
7	8	2	2639459	203406	170426	15.5×
7	8	2	2639459	203406	170426	15.5×
7	4	0	925612	186148	131228	7.1×
7	4	0	925612	186148	131228	7.1×
7	2	0	178080	183477	117573	1.5×
7	2	0	178080	183477	117573	1.5×
11	23	9	19748700	455268	455106	43.4×
11	23	9	19748700	455268	455106	43.4×
11	11	2	9068321	389880	353179	25.7×
11	11	3	9083208	397607	360888	25.2×
11	5	1	5206542	376747	325398	16×
11	5	1	5206542	376747	325398	16×
11	2	0	3560612	365589	306925	11.6×
11	2	0	3560612	365589	306925	11.6×
13	29	17	42651459	653941	653635	65.3×
13	29	17	42651459	653941	653635	65.3×
13	14	6	23810227	530356	486250	49×
13	14	7	23825114	538083	493959	48.2×
13	7	0	9034638	462919	367590	24.6×
13	7	1	9049525	470646	375299	24.1×
13	3	0	1940845	449173	324512	6×
13	3	0	1940845	449594	324933	6×

Where the remaining cost is, and when to prune. Pinning every input so one assignment remains isolates the fixed per-call cost on the combined network: d = 3: 83862 charged steps with the crate's pruning plan, 12907 without, on 16905 edges; d = 4: 139761 charged steps with the crate's pruning plan, 17403 without, on 29329 edges. Building the pruning plan is about 85% of the overhead (about 4 charged steps per edge). With the derived network's search space halved, the plan can cost more than it saves: switching pruning off (the public `Counter::functional_pruning` field) keeps every count exact on all networks (`true`) and makes the combined network 1.21–2.92× cheaper at d = 3–4, for 3–80.2× below the pruned original. The original network needs its pruning (off costs it 1.47–35.6×). The advantage shrinks with d, because the search space grows 4× per bit while the plan grows with network size:

p	d	τ	free bits	pruned / unpruned steps	faster
17	5	37	10	1.17	pruning off
17	5	4	10	1.03	pruning off
37	6	79	12	1.04	pruning off
37	6	9	12	0.85	pruning on

So the plan should be built only when the search space is large relative to the network: off is better through d = 5 here, and at d = 6 pruning already wins at a small threshold. A size-based rule, not a blanket switch.

Dropping a duplicated validity check ("lean"). `input_point` validates A_1 (canonical coordinates, on the curve), and `point_add` then validates $-A_1$, which holds exactly when A_1 is valid (same x; y made canonical by `modular_subtract`; $(-y)^2 = y^2$). Building A_1 from raw input words removes the first check: d = 3: 16905 ⟶ 14340 edges, d = 3: 16987 ⟶ 14418 edges, d = 4: 29329 ⟶ 24540 edges. On 32 instances every count stays exact with and without pruning and every witness checks out (**`true`**). The effect depends on the threshold: at $\tau \le s/4$ lean is 1.08–1.66× faster (the hash rejects most A_1 before any curve check runs), at $\tau \ge s/2$ it is slower (0.59–0.91×), because the curve check, which rejects most (x, y) pairs, now runs only inside `point_add`. Real factor bases use tiny thresholds (at ECCp-131 about 2^{65} of 2^{131} points, $\tau/s \approx 2^{-66}$), which is the lean regime; like the pruning plan, it is a threshold-based choice.

At ECCp-131 the exact fallback's worst case drops from 2^{524} to 2^{262} assignments per count. That is a large constant-exponent improvement and still far beyond Pollard rho's $2^{65.3}$ group operations for the whole problem; it does not change the conclusion below.

The recursion crossover (headline)

The only part of the algorithm that could beat exponential cost is the recursive backend (OpenAI Appendix D.4). In the crate's `Counter::count`, it is selected only when root_u $= \lambda/131072$ is at least 1, with $\lambda = \lceil \log_2(2 \cdot \text{edges}) \rceil$. That needs $\lambda \ge 131072$, i.e. at least 2^{131070} edges (further gates only raise it).

The largest network built here is $2^{20.7}$ edges. The gap to the crossover is about 2^{131049}. Across all 16 counts and 3 solves the recursive backend was selected **`false`** times: every exact count used the fallback (`Functional`/`Direct`), whose cost is $2^{\text{free input bits}}$ per contraction.

This threshold is faithful to the manuscript, which states a saving only above networks of size 2^17 = 131072 (manuscript's 2^(2^17)). No prime-field curve produces a network near it, so this implementation gives no sub-exponential behaviour at any reachable size, including ECCp-131.

Derived: ECCp-131

The backend actually used at every measured size is `Functional`, which visits $2^{\text{free input bits}}$ assignments. With two slots at d = 131 that is 2^{524} assignments per contraction (Functional backend visits 2^(free input bits); measured equal to 2^(4d) at d=3,4; two slots at d=131 give 4*131 = 524 free bits). With A_2 derived (above) it is 2^{262}. Pollard rho on ECCp-131 needs about $2^{65.33}$ group operations for the whole discrete logarithm. The recursive backend did not run at any measured size; this report says nothing about its cost.

What this does not show

No ECCp-131 scalar is computed or attempted, and nothing here approaches Pollard rho. The fallback speed-ups above are measured in the crate's charged steps on small curves, against the same code's earlier behaviour; they are not wall-clock benchmarks and not a comparison with rho. The construction and primality timings are unisolated diagnostics. The recursive backend's asymptotic bound (Bochum v0.2, conditional on OpenAI's unreviewed Theorem D.1) is not tested. No row is added to `benchmarks/README.md`: the validation gate admits no row without an independent replay certificate, validator report and matched rho reference.